A Project Report on

AI-Driven Loan Eligibility Prediction and Document Verification System

Submitted in partial fulfillment for award of

Bachelor of Technology

Degree

in

CSE (Artificial Intelligence and Machine Learning)

By

B.CHAITANYA SRI (Y22ACM408) **D.NAGA LAKSHMI (Y22ACM427)**

B.RAVINDRA (Y22ACM410) **K.PEERAIAH (Y22ACM436)**

Under the guidance of

Dwija Desireddy, M. Tech

Assistant Professor

Department of CSE (Artificial Intelligence and Machine Learning)

Bapatla Engineering College

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522 102, Andhra Pradesh, INDIA

2025-2026

Department of
CSE (Artificial Intelligence and Machine Learning)

CERTIFICATE

This is to certify that the project report entitled "**AI-Driven Loan Eligibility Prediction and Document Verification System**" that is being submitted by **BATTA CHAITANYA SRI (Y22ACM408), DONTIBOINA NAGA LAKSHMI (Y22ACM427), BELLAM RAVINDRA (Y22ACM410), KALAGOTLA PEERAIAH (Y22ACM436)** in partial fulfillment for the award of the Degree of Bachelor of Technology in CSE (Artificial Intelligence and Machine Learning) to the Acharya Nagarjuna University is a record of Bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide	**Signature of the HOD**
Dwija Desireddy	**Dr.K.ManiDeep**
Assistant Professor	**Assoc. Prof. & Head**

DECLARATION

We declare that this project work is composed of ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

B.Chaitanya Sri (Y22ACM408)
D.Naga Lakshmi (Y22ACM427)
B.Ravindra (Y22ACM410)
K.Peeraiah (Y22ACM436)

Acknowledgement

We sincerely thank the following distinguished personalities who have given their advice and support for successful completion of the work.

We are deeply indebted to our most respected guide **Ms.Dwija Desireddy**,Assistant Professor, Department of CSE (Artificial Intelligence and Machine Learning), for her valuable and inspiring guidance, comments, suggestions and encouragement.

We extend our sincere thanks to **Dr. K. Mani Deep**,Associate Professor, Head of the Department of CSE(Artificial Intelligence and Machine Learning), for extending his cooperation and providing the required resources.

We would like to thank our beloved Principal **Dr. B.Srinivasa Rao**,Associate Professor for providing the online resources and other facilities to carry out this work.

We would like to sincerely thank our project coordinator **Ms. G.V. Leela Kumari**, Assistant Professor, Department of CSE(Artificial Intelligence and Machine Learning), for her helpful suggestions in presenting this document.

We extend our sincere thanks to all other teaching faculty and non-teaching staff of the department, who helped directly or indirectly with their cooperation and encouragement.

B.Chaitanya Sri (Y22ACM408)
D.Naga Lakshmi (Y22ACM427)
B.Ravindra (Y22ACM410)
K.Peeraiah (Y22ACM436)

ABSTRACT

Loan processing in banks involves evaluating an applicant's financial details and checking whether the submitted documents are valid. In many cases, this verification is still done manually, which can slow down the approval process and even create errors. In this study, we present a hybrid system that integrates document verification with AI-based loan evaluation and monitoring methods. offers a quick explanation for the decision, adding transparency and improving the efficiency of the process. Applicants fill in their financial details and upload documents, including identity proof, salary slips, and credit reports. The system analyzes these documents to extract relevant information and compares it with what the applicant has filled out; any discrepancies should raise a red flag, which is done through LlamaIndex. After verifying the data, we use a LoRA fine-tuned Qwen model to analyze the applicant's financial profile and predict whether the loan is approved or rejected. The system also provides a brief explanation for the decision, making the process more transparent and efficient.

Keywords:

Loan Eligibility Prediction, Document Verification, LlamaParse, LlamaIndex, Qwen LLM, LoRA Fine-Tuning, Explainable AI, CIBIL Score, Flask, Financial Analysis, Automated Decision System.

TABLE OF CONTENTS

CONTENTS **PAGE NO**

ABSTRACT ..... **V**

LIST OF TABLES ..... **VIII**

LIST OF FIGURES ..... **IX**

LIST OF ABBREVIATIONS ..... **X**

1. INTRODUCTION **1**

1.1. Background of the Study 1

1.2. Problem Statement 2

1.3. Objectives of the Project 3

1.4. Scope of the Project 4

1.5. Organisation of the Report 5

2. LITERATURE REVIEW **7**

2.1 Overview of Existing Work 7

2.2 Comparison of Previous Approaches 8

3. SYSTEM ANALYSIS **10**

3.1. Existing System 10

3.2. Proposed System 10

3.3. Feasibility Study 11

3.4. Requirements 12

4. SOFTWARE DESIGN **15**

4.1. Architecture Design 15

4.2. UML Diagrams 15

4.3. Database Design 17

4.4. Algorithm / Design Methodology 18

5. IMPLEMENTATION **22**

5.1. Technologies Used 22

5.2. Modules Description 23

5.3. Code Snippets 23

6. TESTING.....42

6.1. Test Cases 42

6.2. Testing Methods.....44

6.3. Results and analysis……45

7. RESULTS AND DISCUSSION 47

7.1. Output Screenshots 47

7.2. Performance Analysis 48

7.3. Comparison with Existing System.....49

8. CONCLUSION AND FUTURE WORK 50

8.1. Summary of the Project 50

8.2. Achievements.....50

8.3. Limitations……51

8.4. Future Enhancements……51

9. REFERENCES……52

LIST OF TABLES

S. No	Tables	Page No
Table 2.1	Literature Survey – Comparative Analysis	8
Table 4.1	Applicant Schema	17
Table 4.2	Document Schema	18
Table 4.3	Input Features Collected from Applicant	19
Table 4.4	Document Types and Extracted Fields	19
Table 4.5	EMI Repayment Capacity Categories	20
Table 4.6	CIBIL Score Band Classification	20
Table 7.1	Comparison Of Traditional and Proposed Study Planning System	49

LIST OF FIGURES

S. No	List of Figures	Page No
Fig 4.1	Proposed System Architecture of AI-Driven Loan Eligibility Prediction	15
Fig 4.2	Use Case Diagram for AI-Driven Loan Eligibility System	16
Fig 4.3	Data Flow Diagram showing Applicant Input, Document Parsing, and AI Decision Flow	17
Fig 4.4	Qwen Transformer Architecture with LoRA Adaptation Layers	21
Fig 6.1	Test Case-1 Output	42
Fig 6.2	Test Case-2 Output	43
Fig 6.3	Test Case-3 Output	44
I/O 7.1	Step 1: Applicant Details Entry Interface	47
I/O 7.2	Step 2: Document Upload and Verification Interface	48
I/O 7.3	Loan Prediction Result with AI Reasoning – LOAN APPROVED	48

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
ML	Machine Learning
LLM	Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition
API	Application Programming Interface
LoRA	Low-Rank Adaptation
CIBIL	Credit Information Bureau (India) Limited
EMI	Equated Monthly Installment
UUID	Universally Unique Identifier
UI	User Interface
UML	Unified Modeling Language
DFD	Data Flow Diagram
PDF	Portable Document Format
NBFC	Non-Banking Financial Company
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
JSON	JavaScript Object Notation
GPU	Graphics Processing Unit
RAM	Random Access Memory
SVM	Support Vector Machine
BERT	Bidirectional Encoder Representations from Transformers
METEOR	Metric for Evaluation of Translation with Explicit ORdering
ROUGE-L	Recall-Oriented Understudy for Gisting Evaluation – Longest Common Subsequence
MoE	Mixture of Experts
FLASK	Flask Web Framework (Python)

CHAPTER-1
INTRODUCTION

1.1 Background of the Study

A Loan Eligibility Prediction System is an intelligent application designed to support financial institutions in evaluating loan applications efficiently and accurately. With rapid advances in Artificial Intelligence, Machine Learning, and document understanding technologies, these systems have evolved from simple rule-based credit scoring tools into sophisticated AI-powered platforms capable of processing complex financial profiles and supporting transparent, explainable lending decisions.

AI-powered loan assessment systems act as decision-support tools rather than replacements for financial officers. They assist institutions by automating document verification, performing financial analysis, predicting creditworthiness, and generating human-readable explanations for each decision. This reduces the manual workload on loan officers and significantly improves the speed, consistency, and fairness of the loan approval process.

1.2 Problem Statement

1.2.1 Rule-Based Credit Scoring Systems

Rule-based systems use predefined financial criteria and fixed scoring formulas to assess loan eligibility. Traditional credit scores such as the CIBIL score in India are computed from payment history, outstanding debt, and credit utilization. While standardized and auditable, these systems cannot adapt to evolving financial patterns and fail to process unstructured document data.

1.2.2. Machine Learning-Based Prediction Systems

Machine learning systems apply classification algorithms such as Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting to predict loan eligibility from structured financial features. These models improve accuracy over rule-based systems but produce black-box decisions with no natural-language explanation.

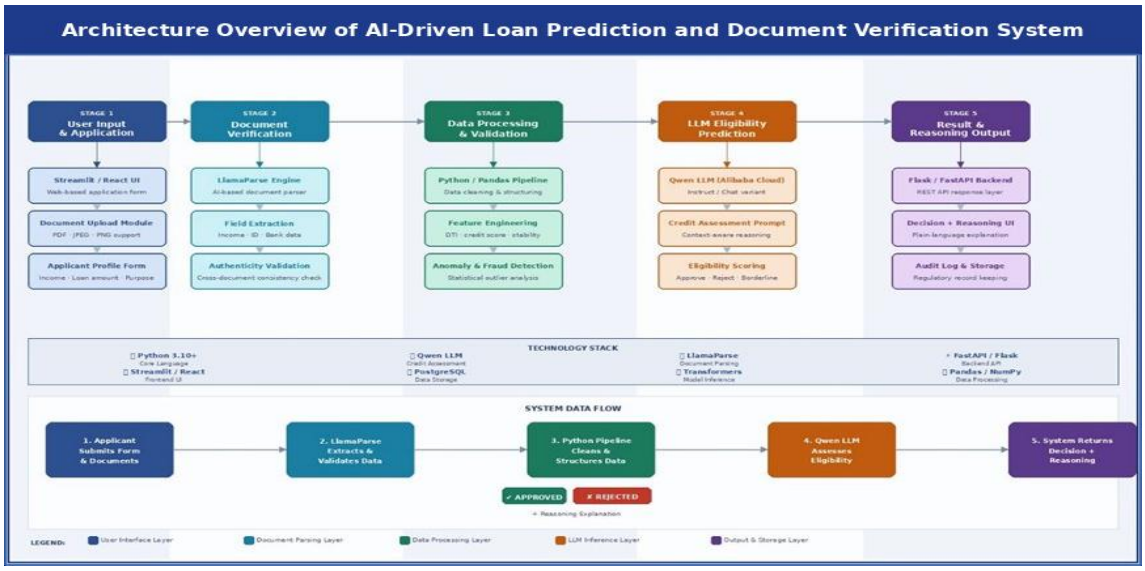


Figure 1.1. Architecture Overview of AI-Driven Loan Prediction and Document Verification System

1.2.3. Hybrid AI Loan Assessment Systems

Hybrid systems combine rule-based financial criteria with AI-based predictions to achieve both regulatory compliance and intelligent reasoning. The proposed system follows this hybrid architecture, integrating structured financial analysis with a LoRA fine-tuned Qwen LLM.

1.2.4. Document-Integrated Assessment Systems

These systems incorporate automated document parsing to extract information directly from uploaded financial documents. By combining document verification with AI prediction, they eliminate the need for manual data entry and reduce fraud risk. The proposed system belongs to this category.

1.2.5. Explainable AI-Based Loan Systems

Explainable AI systems generate human-readable reasoning for every loan decision. Unlike black-box classifiers, they provide applicants and loan officers with a transparent account of the factors driving the eligibility outcome, improving trust and supporting regulatory compliance.

1.2.6. Large Language Model-Based Financial Decision Systems

LLM-based systems use transformer models such as GPT-4, LLaMA, and Qwen to reason over complex financial profiles and generate contextually aware lending decisions. The proposed system uses a LoRA fine-tuned Qwen LLM as its core prediction engine, enabling both accurate decisions and natural-language explanations.

1.3 Objectives of the Project

The primary aim of the AI-Driven Loan Eligibility Prediction and Document Verification System is to design and develop an end-to-end intelligent platform that automates the complete loan evaluation process for financial institutions. The system aims to eliminate the inefficiencies and inaccuracies of manual document review, apply structured financial analysis, and leverage a fine-tuned large language model to produce transparent, explainable lending decisions.

The project aims to utilise intelligent document parsing through LlamaParse to automatically extract verified financial data from uploaded documents, and apply LoRA fine-tuning to adapt the Qwen LLM for accurate domain-specific loan eligibility prediction. The overall aim is to deliver a reliable, fast, and transparent decision-support system that reduces processing time from days to seconds while improving the consistency and explainability of loan decisions.

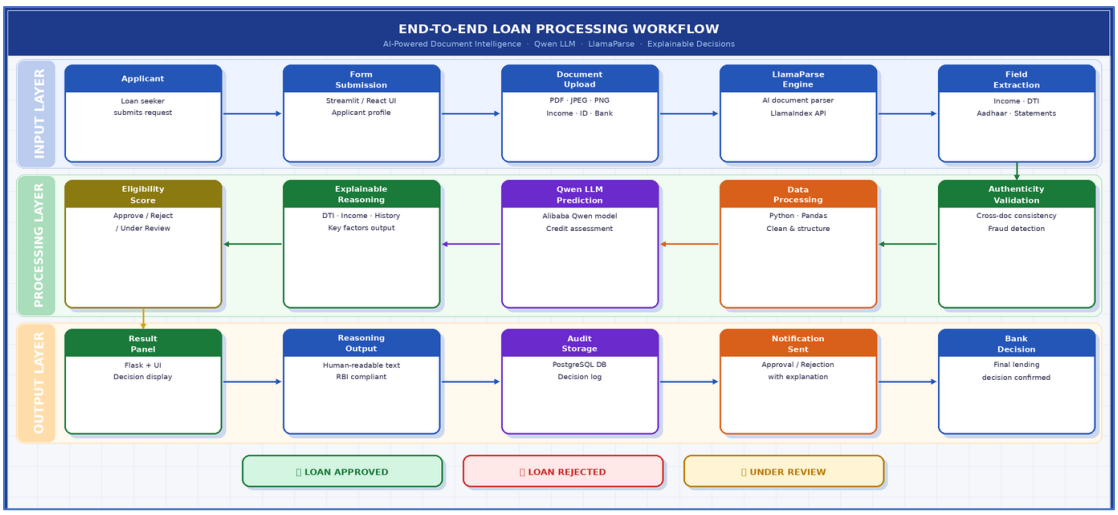


Figure 1.2. End-to-End Loan Processing Workflow with AI and Document Intelligence

1.4 Scope of the Project

1.4.1. Automated Loan Application Processing

The system processes complete loan applications automatically, from initial data collection through document verification, financial analysis, and AI-based eligibility prediction, without requiring manual intervention by loan officers at any stage.

1.4.2. Intelligent Document Verification

The scope includes automated parsing and cross-verification of three document types: government identity proof, salary slip, and CIBIL credit report. LlamaParse extracts key fields from diverse document formats and compares them against applicant-submitted values to detect discrepancies.

1.4.3. Financial Risk Metric Computation

The system computes derived financial indicators including total income, usable income, estimated EMI, EMI-to-income ratio, and CIBIL score band to provide a structured financial health assessment for each applicant.

1.4.4. Explainable AI Decision Generation

The scope includes the generation of a natural-language reasoning explanation for every loan eligibility decision, making the system's output transparent and understandable to both applicants and financial officers.

1.4.5. Fraud Risk Detection

By cross-verifying document-extracted values against form-submitted data, the system identifies potential misrepresentations and assigns a document verification risk level (Verified, Minor Discrepancy, or Major Discrepancy) that influences the final AI decision.

1.5 Organisation of the Report

1.5.1. Manual Processing Bottlenecks

Most existing loan assessment processes rely on manual document review by loan officers, resulting in processing times of several days to weeks. This creates bottlenecks for both applicants awaiting decisions and institutions managing large volumes of applications.

1.5.2. Lack of Document Verification in ML Systems

Existing ML-based loan prediction systems process only structured tabular data entered manually by loan officers. They do not automate the verification of uploaded financial documents, creating a significant gap between automated prediction and real-world application requirements.

1.5.3. Black-Box Decision Making

Traditional ML classifiers including Logistic Regression, Random Forest, and neural networks produce only a binary Approved or Rejected output without any explanation.

Applicants who are rejected cannot understand the reason for the decision, and institutions cannot demonstrate regulatory compliance with explainability requirements.

1.5.4. Fragmented Pipeline

Existing systems address document processing and loan prediction as separate problems. No existing system integrates intelligent document parsing, automated cross-verification, structured financial analysis, and explainable LLM-based prediction in a single unified pipeline.

1.5.5. Inconsistent Decision Quality

Manual evaluation introduces human bias and inconsistency across different officers or branches. The same financial profile may receive different outcomes depending on the reviewing officer, reducing the fairness and reliability of the lending process.

CHAPTER-2
LITERATURE REVIEW

2.1.OVERVIEW OF EXISTING WORK

Artificial Intelligence (AI) has played a transformative role in modern education by enabling personalized learning and intelligent academic support systems. Several studies have examined different aspects of AI in education. For instance, adaptive learning platforms and AI-driven educational software analyze learner interactions and performance patterns to deliver customized learning experiences and instructional recommendations [2]. Deep learning techniques have been applied in adaptive e-learning environments to monitor student activities and engagement in real time, thereby improving the understanding of learner behavior and participation [3]. Technology-enhanced learning environments, such as robotics-based educational tools, further support the development of cognitive skills and self-regulated learning among students [4]. In addition, data mining and big data analytics have been utilized to analyze large-scale educational datasets, identifying patterns that aid predictive decision-making in academic contexts [5]. Productivity and engagement applications have also been introduced to help students monitor study habits and balance academic tasks with daily routines [6]. Deep neural network models have been employed in real-time monitoring systems to detect and analyze behavioral patterns [7], while AI-based personalized learning platforms dynamically adapt instructional paths and materials based on student progress and performance [8].

Despite these advancements, most existing systems primarily emphasize personalized content delivery, learner engagement monitoring, or educational data analysis, rather than intelligent study scheduling and structured academic task management. Current tools often rely on static scheduling mechanisms or domain-specific adaptive learning approaches that fail to dynamically adjust study plans according to student productivity patterns, workload distribution, or learning behavior. Recent research has proposed AI-powered smart study planners that integrate machine learning algorithms, behavioral analytics, and adaptive scheduling techniques to generate personalized study schedules tailored to learner preferences, goals, and performance data. These systems have shown improvements in learning efficiency and academic engagement compared to traditional

manual planning methods. However, there remains a need for more comprehensive frameworks that effectively combine behavioral analysis, adaptive scheduling, and intelligent workload management into a unified academic support system [9].

2.2 COMPARISON OF PREVIOUS APPROACHES

Table 2.1: Literature Survey – Comparative Analysis

Author & Title	Models / Tools Used	Advantages	Disadvantages
Bansal et al. [1] 2020	• Logistic Regression • Decision Tree • Random Forest	• Ensemble methods outperform single classifiers • Accuracy above 80% • Simple and interpretable pipeline	• Structured inputs only; no document verification • Requires manual data entry • Unsuitable for document-based deployment
Naik [2] 2021	• LightGBM • XGBoost • Random Forest	• Superior speed and accuracy vs XGBoost/RF • Handles large-scale datasets efficiently • Strong predictive power via gradient boosting	• Works only on numerical/categorical features • No document parsing or verification • No natural-language explanation
Liu et al. [3] 2022	• Deep Neural Network	• Captures complex feature interactions • Better accuracy than traditional ML • Scales well with data volume	• Computationally intensive; needs large datasets • Black-box; no explainability mechanism • No document parsing or verification
Jajoo et al. [4] MASCA	• Multi-Agent LLM • Specialized agents	• Multi-agent collaboration enhances scoring • LLMs bring reasoning over financial text • Modular agent design improves coverage	• Focuses only on structured financial data • No automated document verification • Cannot process uploaded physical documents
Brown & Mues [5] 2012	• Logistic Regression • SVM • Neural Networks • Ensemble methods	• Comprehensive classifier comparison • Highlights class imbalance impact • Provides imbalance handling best practices	• Applied to structured tabular data only • No document verification integration • No natural-language decision explanation
LlamaIndex [6] LlamaParse	• LlamaParse • OCR + LLM extraction	• Processes PDFs, scanned images, layouts • No predefined templates needed • State-of-the-art document extraction	• Not integrated with loan decision logic • Accuracy depends on document quality • No built-in credit risk model

Author & Title	Models / Tools Used	Advantages	Disadvantages
Hu et al. [7] 2022	• LoRA • Transformer fine-tuning	• Parameter-efficient fine-tuning • Standard method for LLM adaptation • Preserves base model knowledge	• Requires curated domain-specific data • Rank selection affects model quality • Not a standalone prediction system
Alibaba Cloud [8] Qwen LLM	• Qwen LLM • Transformer (multilingual)	• Strong reasoning on financial inputs • Multilingual support for applicant data • Well-suited for domain tasks with LoRA	• Requires fine-tuning for finance tasks • Large model demands significant compute • Base model alone insufficient for loans

The literature review identifies a clear and consistent research gap: no existing system presents a unified end-to-end pipeline that integrates automated document verification with LLM-based explainable loan eligibility prediction. The proposed system bridges this gap by combining LlamaParse for document parsing and cross-verification with a LoRA fine-tuned Qwen LLM for transparent loan eligibility assessment.

CHAPTER-3
SYSTEM ANALYSIS

3.1 Existing System

Financial institutions process thousands of loan applications every month, each requiring document collection, verification, financial analysis, and eligibility assessment. This process is predominantly manual in most institutions, resulting in processing delays of several days to weeks, high operational costs, inconsistent decision quality across reviewing officers, and a complete absence of transparent justifications for rejected applications. The increasing volume of loan applications further strains the capacity of manual evaluation processes without a proportional increase in staffing.

Recent advances in artificial intelligence, intelligent document parsing, and large language models present an opportunity to automate the complete loan evaluation pipeline, combining document verification with explainable AI-based prediction in a single integrated system. The absence of such a unified system in current practice constitutes the core problem addressed by this project.

3.2 Proposed System

The key challenges motivating this research can be summarized as follows:

- Absence of automated document verification in existing ML-based loan prediction systems, requiring manual document review by loan officers for every application.
- Lack of integrated platforms capable of combining document parsing, cross-verification, financial analysis, and AI-based prediction within a single processing pipeline.

- Black-box decision-making in traditional ML classifiers, with no natural-language explanation provided for approved or rejected loan decisions.
- High processing times in manual evaluation workflows, with loan decisions typically requiring several days or weeks to complete.
- Fraud risk from unverified discrepancies between applicant-submitted data and official document values, which manual review often fails to detect.
- Inconsistent application of evaluation criteria across different reviewing officers and branches, reducing fairness and reliability of lending decisions.
- Regulatory pressure on financial institutions to document and justify their lending decisions, which opaque ML models cannot satisfy.

3.3 Feasibility Study

The feasibility study evaluates the practicality and viability of implementing the proposed AI-Driven Loan Eligibility Prediction and Document Verification System.

3.3.1 Technical Feasibility

The system is technically feasible as it is built using a robust and modern technology stack. The backend is powered by Python 3.10+ with Flask for HTTP routing, session management, and pipeline coordination. Document parsing is handled by LlamaParse (LlamaIndex), which intelligently extracts structured fields from uploaded financial documents using OCR and LLM-based techniques. Loan eligibility decisions are generated by a LoRA fine-tuned Qwen LLM, loaded via Hugging Face Transformers with PEFT adapter support, ensuring efficient domain adaptation. Financial metrics such as Debt-to-Income (DTI) ratio and Equated Monthly Installment (EMI) are computed using Pandas and NumPy. The frontend is developed with HTML5, CSS3, and JavaScript, providing an interactive and responsive user interface. The system requires a minimum of 16 GB RAM and an NVIDIA CUDA-capable GPU to support efficient LLM inference, making it technically achievable with current computing resources.

3.3.2 Economic Feasibility

The project is economically feasible since it leverages open-source frameworks such as Flask, Hugging Face Transformers, Pandas, and NumPy, which eliminate the need for costly proprietary licenses. Deployment can be done on standard cloud platforms, with scalable GPU resources provisioned only when required. Maintenance costs are minimized due to the modular architecture and reliance on widely adopted open-source tools.

3.3.3 Operational Feasibility

The system is operationally feasible as it is designed with a simple, user-friendly interface accessible to loan officers and applicants. Automated document parsing reduces manual workload, while AI-driven predictions provide accurate eligibility assessments with reasoning explanations. The workflow aligns with existing loan processing practices, making adoption practical in real-world financial institutions.

3.4 Requirements

3.4.1 Hardware Requirements

The system requires moderate hardware for deployment and higher specifications for AI model training and inference:

- **Processor:** Intel Core i5 or higher
- **RAM:** Minimum 16 GB (recommended for LLM inference)
- **Storage:** Minimum 50 GB available disk space
- **GPU:** NVIDIA CUDA-capable GPU (e.g., RTX series) for efficient LLM inference
- **Internet Connection:** Stable broadband connection
- **Display:** Standard monitor with resolution of 1366 × 768 or higher

3.4.2 Software Requirements

The system is developed using modern AI and web technologies. Required components include:

- Operating System: Windows / Linux / macOS
- Backend Framework: Flask (Python 3.10+)
- Frontend Technologies: HTML5, CSS3, JavaScript
- AI Libraries: Hugging Face Transformers, PEFT (LoRA), Pandas, NumPy

- Document Parsing: LlamaParse (LlamaIndex)
- Runtime Environment: Python environment with CUDA support
- Code Editor: Visual Studio Code / PyCharm
- Web Browser: Google Chrome / Mozilla Firefox
- Version Control System: Git

3.4.3 Functional Requirements

The system shall:

- Allow applicants to upload financial documents (salary slips, ID proofs, credit reports).
- Parse and extract structured fields using LlamaParse.
- Compute financial metrics such as DTI ratio and EMI.
- Predict loan eligibility using LoRA fine-tuned Qwen LLM.
- Provide reasoning explanations for AI-driven predictions.
- Allow loan officers to review parsed documents and AI-generated eligibility reports.
- Maintain applicant history and decision logs.
- Support multilingual inputs for diverse applicants.
- Provide a dashboard interface for monitoring applications and decisions.

3.4.4 Non-Functional Requirements

- Performance: The system shall process documents and generate eligibility predictions within acceptable response times.
- Usability: The interface shall be intuitive, responsive, and accessible to both loan officers and applicants.
- Reliability: The system shall ensure stable performance and consistent predictions.
- Scalability: The architecture shall support increasing applicant volumes and integration with multiple financial institutions.
- Maintainability: The codebase shall follow modular design principles for easy updates and enhancements.
- Security: Strong encryption, input validation, and secure data handling shall be implemented to protect sensitive applicant information.

CHAPTER 4
SOFTWARE DESIGN

4.1 ARCHITECTURE DESIGN

The architecture of the proposed AI-Driven Loan Eligibility Prediction and Document Verification System is designed using a modular layered structure to ensure scalability, maintainability, and efficient interaction among components. The system begins with the User Input Module, where applicants provide personal and financial details along with document uploads. These documents are processed in the Document Processing Module using LlamaParse to extract structured data such as identity, income, and CIBIL score. The Data Preprocessing Module then performs profile transformation, EMI ratio calculation, repayment capacity assessment, and document verification to ensure accuracy. The verified data is passed to the LLM Decision Engine, where a LoRA fine-tuned Qwen model constructs prompts and performs inference to predict loan eligibility. Finally, the Output and Result Display Module presents the decision (Approved/Rejected) along with AI-generated reasoning, ensuring transparency and usability for applicants checking their eligibility.

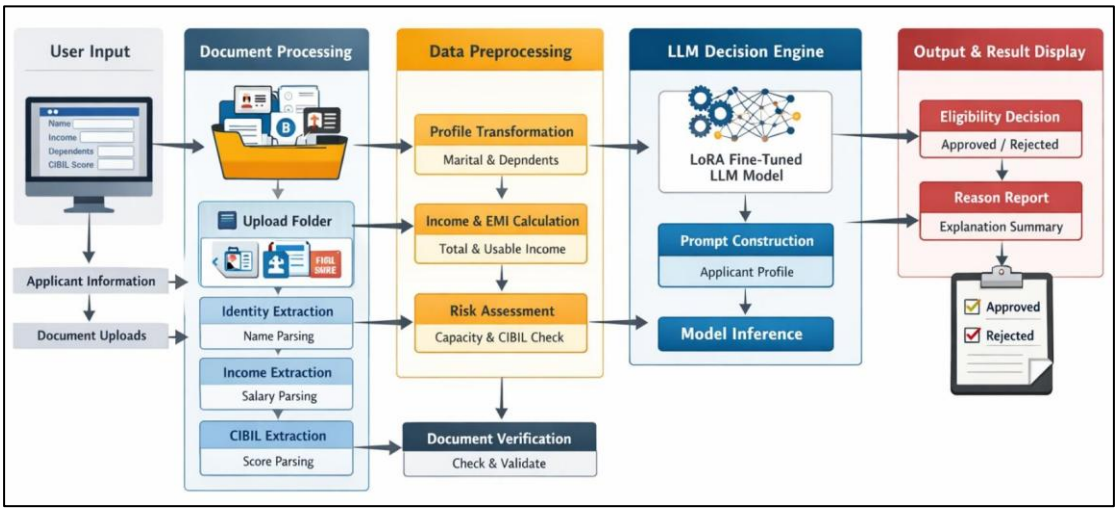


Figure 4.1. Proposed System Architecture of AI-Driven Loan Eligibility Prediction

4.2 Unified Modeling Language (UML)

Unified Modeling Language (UML) is a standardized graphical modeling language used to visualise, specify, design, and document the structure and behaviour of a software system. UML diagrams help represent system components, interactions, workflows, and data flows in a clear and organized manner.

Types of UML Diagrams:

- Use Case Diagram
- Data Flow Diagram (DFD)

4.2.1 Use Case Diagram

The Use Case Diagram represents the functional interactions between actors and the AI-Driven Loan Eligibility Prediction System. The primary actor is the Loan Applicant, who accesses the system through the web interface to submit an application and receive an eligibility decision. The Applicant can perform actions including entering personal and financial details, uploading supporting documents, and viewing the AI-generated loan decision and reasoning explanation. The Flask Backend acts as the system controller, coordinating data flow between the frontend and the processing modules. LlamaParse acts as an external service actor that performs document parsing. The LoRA fine-tuned Qwen LLM acts as the AI inference actor that generates the loan decision.

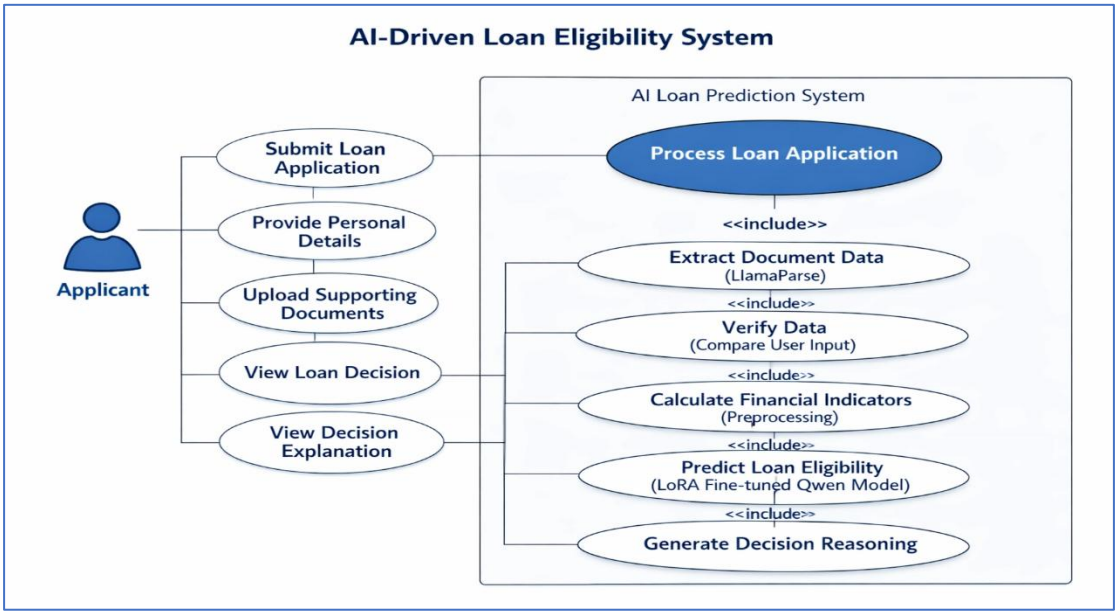


Figure 4.2. Use Case Diagram for AI-Driven Loan Eligibility System

4.2.2 Data Flow Diagram

The Data Flow Diagram (DFD) represents the complete flow of information within the AI-Driven Loan Eligibility Prediction System. Applicant inputs including personal details, financial information, and uploaded documents are collected through the HTML web interface. The Flask backend routes the form data and uploaded files to the processing pipeline. Uploaded documents are processed by LlamaParse to extract structured fields including name, income, and CIBIL score. The extracted values flow

into the verification module, which compares them against form-submitted data. Verified financial data and derived metrics flow into the Qwen LLM inference module, which processes the structured applicant profile to generate a loan eligibility decision and natural-language reasoning. The final decision and reasoning flow back to the result page for display to the applicant.

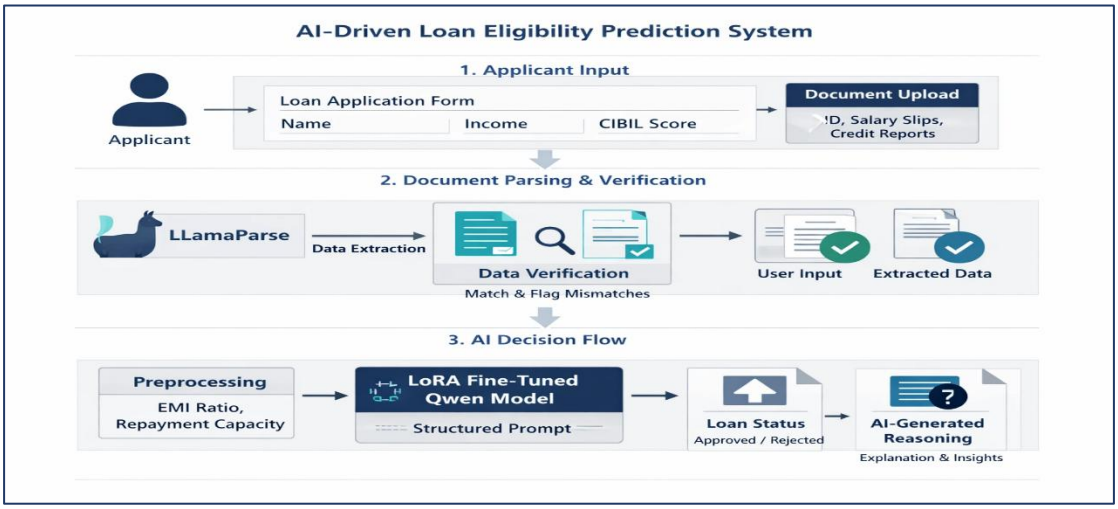


Figure 4.3. Data Flow Diagram showing Applicant Input, Document Parsing, and AI Decision Flow

4.3 Database Design

The proposed AI-Driven Loan Eligibility Prediction and Document Verification System does not rely on a traditional relational database. Instead, the system utilizes a lightweight file-based and structured storage approach to manage applicant data and uploaded documents. This design ensures simplicity, efficiency, and conflict-free management for single-user or prototype deployment.

The primary data entities stored within the system include Applicant Details, Document Records, Verification Logs, and Loan Prediction Results.

Table 4.1 Applicant Schema

Field Name	Data Type	Description
id	String	Unique identifier for applicant
name	String	Applicant full name
income	Number	Monthly income
cibilScore	Number	Credit score extracted or entered
loanAmount	Number	Requested loan amount

loanPeriod	Number	Loan repayment period (months)
dependents	Number	Number of dependents
employmentStatus	String	Employment type (Salaried/Self-employed)
propertyLocation	String	Location of property (Urban/Rural)

Table 4.2 Document Schema

Field Name	**Data Type**	**Description**
docId	String	Unique identifier for document
applicantId	String	Linked applicant identifier
docType	String	Type of document (ID, Salary Slip, CIBIL)
filePath	String	Storage path for uploaded file
parsedValue	String	Extracted structured value from document
verificationFlag	Boolean	Indicates mismatch or successful match

4.4 Algorithm / Design Methodology

The proposed system combines automated document verification with AI-based loan eligibility prediction through a structured five-phase methodology. Each phase is designed to process applicant data accurately, verify document authenticity, compute financial indicators, and generate explainable loan decisions. The methodology supports reliable, transparent, and scalable loan evaluation across diverse applicant profiles.

4.4.1 User Input Module

4.4.1.1 Applicant Information Collection

The first phase collects personal and financial information from the loan applicant through the Step 1 web form interface. The form is implemented in HTML with a two-column layout grouping related fields together. Client-side JavaScript validates all inputs before submission. The collected data is transmitted via HTTP POST to the Flask backend and stored in the session for use in subsequent pipeline stages. Table 7.1 lists all eleven input features collected in this phase.

Table 4.3: Input Features Collected from Applicant

Feature	Data Type	Description
Full Name	Text	Applicant full legal name for identity matching
Gender	Categorical	Male or Female
Marital Status	Categorical	Married or Unmarried
Number of Dependents	Numeric	Count of financial dependents (0, 1, 2, 3+)
Applicant Monthly Income	Numeric	Primary monthly income in INR
Co-Applicant Monthly Income	Numeric	Secondary income in INR (0 if none)
CIBIL Credit Score	Numeric	Credit score on scale 300 to 900
Loan Amount	Numeric	Requested loan principal in INR
Loan Period	Numeric	Requested repayment duration in months
Employment Status	Categorical	Salaried or Self-Employed
Property Location	Categorical	Urban, Semi-Urban, or Rural

4.4.1.2 Document Upload

After completing Step 1, the applicant uploads three documents through the Step 2 interface: government identity proof, salary slip, and CIBIL credit report. Files are accepted in PDF and common image formats. The Flask backend saves each uploaded file to a UUID-named subdirectory under the uploads folder, preventing file naming conflicts between concurrent applications. After all three documents are received, the pipeline proceeds to the document parsing phase.

4.4.2 Document Parsing Module

Table 7.2 summarizes the three document types handled by the system and the key fields extracted from each by LlamaParse.

Table 4.4: Document Types and Extracted Fields

Document Type	Format Accepted	Fields Extracted
Government Identity Proof (Aadhaar / PAN)	PDF / PNG / JPEG	Full name, date of birth, address, ID number
Salary Slip	PDF / PNG / JPEG	Applicant name, employer name, net monthly income, pay period
CIBIL Credit Report	PDF / PNG / JPEG	Applicant name, CIBIL credit score, credit summary

4.4.3 Data Preprocessing and Feature Engineering

4.4.3.1 Income Computation

Total income is computed as the sum of applicant and co-applicant monthly income, as shown in Equation 7.1.

$$\text{Total Income} = \text{Applicant Income} + \text{Co-Applicant Income} \quad \text{... (7.1)}$$

Usable income is then derived using a 50% deduction for living expenses, as shown in Equation 7.2.

$$\text{Usable Income} = \text{Total Income} \times 0.5 \quad \text{... (7.2)}$$

4.4.3.2 EMI and Repayment Capacity

The estimated monthly EMI is computed using the standard amortisation formula (Equation 7.3), where P is the principal, r is the monthly interest rate (10% per annum / 12), and n is the number of months.

$$\text{EMI} = P \times r \times (1 + r)^n / ((1 + r)^n - 1) \quad \text{... (7.3)}$$

The EMI-to-income ratio is computed using Equation 7.4.

$$\text{EMI Ratio} = \text{Estimated EMI} / \text{Total Income} \quad \text{... (7.4)}$$

Table 4.5: EMI Repayment Capacity Categories

EMI-to-Income Ratio	Repayment Capacity	Risk Level
Less than 0.30	High	Low Risk
0.30 to 0.50	Moderate	Medium Risk
Greater than 0.50	Low	High Risk

4.4.3.3 CIBIL Score Band Classification

Table 4.6: CIBIL Score Band Classification

CIBIL Score Range	Score Band	Credit Quality
300 – 549	Poor	Very High Default Risk
550 – 649	Fair	High Default Risk
650 – 749	Good	Moderate Default Risk
750 – 900	Excellent	Low Default Risk

4.4.4 LoRA Fine-Tuning of Qwen Model

4.4.4.1 Qwen Model Architecture

The Qwen model is a transformer-based large language model developed by Alibaba Cloud. The architecture processes input text through token embedding, RMS normalisation, multi-head self-attention, mixture-of-experts feed-forward layers, and output linear projection. Each stage performs well-defined mathematical transformations to progressively refine token representations for next-token prediction

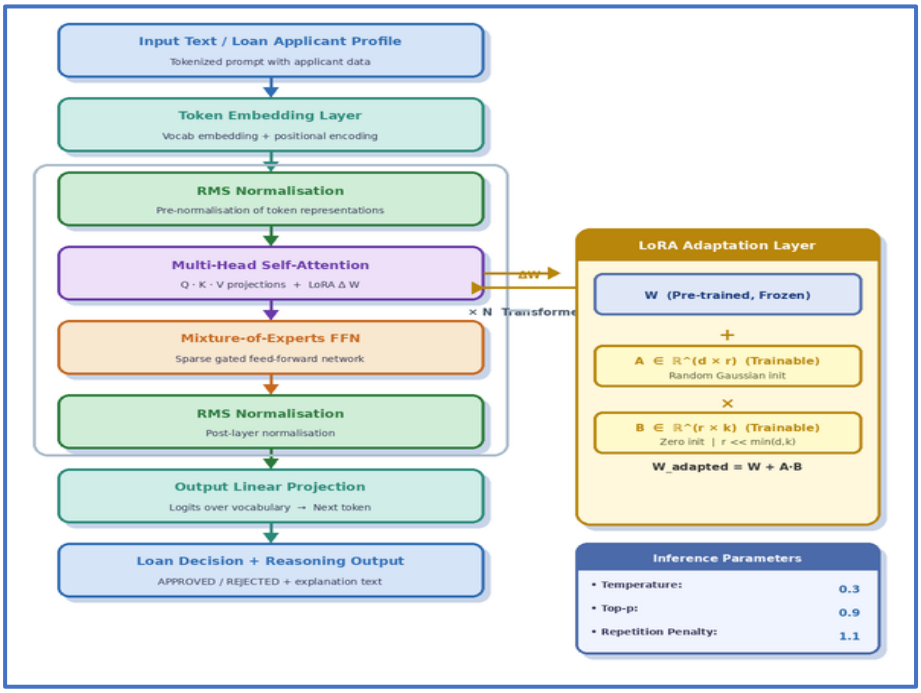


Figure 4.4. Qwen Transformer Architecture with LoRA Adaptation Layers

CHAPTER 5

IMPLEMENTATION

5.1 Technologies Used

The proposed AI-Driven Loan Eligibility Prediction and Document Verification System is developed using modern web technologies and AI-integrated frameworks to ensure scalability, responsiveness, and intelligent functionality. The technologies used in the implementation are described below:

Flask

Flask is used as the backend framework to manage server creation, API routing, and communication between frontend modules and AI service components.

Python

Python serves as the primary programming language for backend development. It enables integration of AI models, document parsing, and financial calculations.

LlamaParse / LlamaIndex

These tools are used for document parsing and verification. They extract structured fields such as applicant name, income, and CIBIL score from uploaded documents and cross-check them against user input.

LoRA Fine-Tuned Qwen LLM

The fine-tuned Qwen model is integrated to provide explainable AI functionalities such as loan approval/rejection reasoning, transparency in decision-making, and natural language explanations.

HTML / CSS / Bootstrap

Used for structuring and styling the frontend interface, ensuring a responsive and user-friendly design.

JavaScript (Fetch API)

Handles smooth step transitions between applicant input, document upload, and result display without requiring full page reloads.

Local Storage / Lightweight File Storage

Used for storing applicant details, document records, verification logs, and prediction results in a simple and efficient manner.

Visual Studio Code

Utilized as the integrated development environment (IDE) for coding, debugging, and project management.

5.2 Modules Description

5.2.1 Applicant Management Component

This module collects and stores applicant details such as name, income, dependents, CIBIL score, loan amount, and employment status. It ensures that data is maintained in a standard format for use by other modules. Functions include adding, updating, and validating applicant information.

5.2.2 Document Processing Component

Handles document uploads including government ID, salary slip, and credit report. LlamaParse extracts structured values from these documents, while LlamaIndex verifies them against user-entered data. Any mismatch is flagged for review.

5.2.3 Loan Calculation Component (Core Intelligence Engine)

This module performs financial calculations such as EMI ratio, repayment capacity, and risk assessment. It evaluates applicant eligibility based on income, dependents, and credit score. Tasks with higher repayment capacity and strong credit history are prioritized for approval.

5.2.4 Verification Engine

Cross-checks extracted document values against applicant input to ensure accuracy. Flags mismatches and strengthens the reliability of the decision-making process.

5.2.5 Generative AI Explanation Component

Enhances transparency by providing natural language reasoning for loan decisions. It explains why an application was approved or rejected and highlights key financial and risk factors.

5.2.6 Closed-Loop Feedback Mechanism

Logs applicant decisions and verification outcomes to improve prediction consistency. Over time, the system adapts to different applicant scenarios, strengthening reliability.

5.2.7 Dashboard and Interaction Component

Provides an interactive interface for users to enter details, upload documents, and view results. Displays applicant history, decision logs, and AI-generated explanations in a clear and user-friendly manner.

5.3 Code Snippets

5.3.1 Project Folder Structure The proposed system is organized into two main parts: **Backend** and **Frontend**.

- **Backend**: Handles AI-related services, document parsing, financial calculations, and API communication.
    - Files: app.py, prediction.py, verification.py
- **Frontend**: Manages the user interface, navigation, applicant input, document upload, and result display.
    - Files: index.html, result.html, styles.css, script.js

5.3.2 Backend Implementation

- **File: app.py** Purpose: Serves as the Flask application backbone. Manages routing, applicant sessions, and API communication. Functions:
    - Accepts applicant details and document uploads
    - Calls prediction engine for loan decision
    - Returns AI-generated reasoning and final decision
- **File: prediction.py** Purpose: Handles AI intelligence tasks including document parsing and loan decision generation. Functions:
    - Extracts structured fields using LlamaParse
    - Computes EMI ratio and repayment capacity
    - Calls Qwen LLM for natural language reasoning
- **File: result.html** Purpose: Displays applicant details, AI reasoning, and final loan decision. Features:
    - Approved/Rejected verdict with distinct color coding
    - Explanation box for transparency
    - Navigation buttons for reapplication or history view

Code:

app.py: This file serves as the backbone of the AI Loan Prediction System, built using the Flask web framework. It manages user sessions, file uploads, and routing across the two-step application process. The model is loaded once at startup to avoid repeated overhead, and each submission triggers document verification, loan prediction, and KPI logging. Error handling is implemented at both the route and global level to ensure graceful failure recovery.

==========

FILE: app.py

==========

```
from flask import Flask, render_template, request, session, jsonify
```

```
import os, uuid, shutil, stat, time, json
import torch
from datetime import datetime
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel
from prediction import get_loan_eligibility_prediction, get_llamaindex_result
# ========================================================
# FLASK APP
# ========================================================
app = Flask(__name__)
app.secret_key = "super-secret-key"
UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

KPI_FILE = "kpi_data.json"
# ========================================================
# MODEL PATHS
# ========================================================
BASE_MODEL_PATH = "Qwen/Qwen2-1.5B"
LORA_PATH = r"c:\Users\bella\Downloads\loan_project\lora_output-
20260309T170107Z-1-001\lora_output"
DEVICE = "cpu"
# ========================================================
# GLOBAL MODEL CACHE
# ========================================================
model = None
tokenizer = None
def load_model_once():
    global model, tokenizer
    if model is not None:
        return model, tokenizer
    print("Loading Qwen + LoRA model (ONE TIME)...")
    tokenizer = AutoTokenizer.from_pretrained(
        BASE_MODEL_PATH,
        trust_remote_code=True
    )
    tokenizer.pad_token = tokenizer.eos_token
    tokenizer.add_special_tokens({"additional_special_tokens":
["<END_OF_DECISION>"]})
    base_model = AutoModelForCausalLM.from_pretrained(
        BASE_MODEL_PATH,
        trust_remote_code=True,
```

```
        dtype=torch.float32,
        low_cpu_mem_usage=True
    )
base_model.resize_token_embeddings(len(tokenizer))
model = PeftModel.from_pretrained(base_model, LORA_PATH)
model.to(DEVICE)
model.eval()
print("Model loaded successfully")
return model, tokenizer
def remove_folder_safely(path):
    def onerror(func, path, exc_info):
        try:
            os.chmod(path, stat.S_IWRITE)
            func(path)
        except Exception:
            pass
    if os.path.exists(path):
        try:
            shutil.rmtree(path, onerror=onerror)
        except Exception:
            time.sleep(0.2)
            shutil.rmtree(path, onerror=onerror, ignore_errors=True)
def loan_user_data_modification(user_data):
    marital_status = "Married" if user_data["marital_status"] == "Married" else
"Unmarried"
    dependents_raw = user_data["dependents"]
    if dependents_raw == "3+" or int(dependents_raw) >= 3:
        dependents = "High"
    elif int(dependents_raw) == 2:
        dependents = "Moderate"
    else:
        dependents = "Low"
    coapplicant_income_raw = float(user_data.get("coapplicant_income") or 0)
    coapplicant_income = "Yes" if coapplicant_income_raw > 0 else "No
    cibil_score = "Good" if int(user_data["cibil_score"]) >= 750 else "Bad"
    total_income = float(user_data["income"]) +
float(user_data.get("coapplicant_income") or 0)
    usable_income = total_income * 0.5
    loan_amount = float(user_data["loan_amount"])
    loan_period = float(user_data["loan_period"])
    emi_ratio = (loan_amount / loan_period) / usable_income if usable_income > 0 else
999
    if emi_ratio <= 1:
```

```
        emi_repayment_capacity = "High"
    elif emi_ratio <= 1.5:
        emi_repayment_capacity = "Moderate"
    else:
        emi_repayment_capacity = "Low"
    return {
        "gender": user_data["gender"],
        "coapplicant_income": coapplicant_income,
        "marital_status": marital_status,
        "dependents": dependents,
        "cibil_score": cibil_score,
        "employment_status": user_data["employment_status"],
        "emi_repayment_capacity": emi_repayment_capacity,
        "place_of_living": user_data["property_location"],
    }
def document_verification(user_data, folder):
    return get_llamaindex_result(user_data, folder)
def record_kpi(user_data, updated_user_data, doc_result, result):
    entry = {
        "timestamp": datetime.now().isoformat(),
        "decision": result["decision"],
        "loan_amount": float(user_data.get("loan_amount", 0)),
        "income": float(user_data.get("income", 0)),
        "coapplicant_income": float(user_data.get("coapplicant_income") or 0),
        "cibil_score": int(user_data.get("cibil_score", 0)),
        "loan_period": int(user_data.get("loan_period", 0)),
        "employment_status": user_data.get("employment_status", ""),
        "gender": user_data.get("gender", ""),
        "property_location": user_data.get("property_location", ""),
        "dependents": user_data.get("dependents", "0"),
        "marital_status": user_data.get("marital_status", ""),
        "document_verification": doc_result.get("document_verification", ""),
        "emi_repayment_capacity": updated_user_data.get("emi_repayment_capacity",
""),
    }
    records = []
    if os.path.exists(KPI_FILE):
        try:
            with open(KPI_FILE, "r") as f:
                records = json.load(f)
        except Exception:
            records = []
    records.append(entry)
```

```
    with open(KPI_FILE, "w") as f:
        json.dump(records, f, indent=2)
@app.route("/")
def home():
    return render_template("index.html")
@app.route("/submit-step1", methods=["POST"])
def submit_step1():
    session["user_data"] = request.form.to_dict()
    session["upload_id"] = str(uuid.uuid4())
    return jsonify({"success": True})
@app.route("/submit-step2", methods=["POST"])
def submit_step2():
    user_data = session.get("user_data")
    upload_id = session.get("upload_id")
    if not user_data or not upload_id:
        return "Session expired. Please start again."
    user_folder = os.path.join(app.config["UPLOAD_FOLDER"], upload_id)
    remove_folder_safely(user_folder)
    os.makedirs(user_folder, exist_ok=True)
    for key in ["gov_id", "salary_slip", "cibil"]:
        file = request.files.get(key)
        if file:
            file.save(os.path.join(user_folder, file.filename))
    updated_user_data = loan_user_data_modification(user_data)
    try:
        doc_result = document_verification(user_data, user_folder)
    except Exception as e:
        print(f"[WARN] Document verification failed: {e}")
        doc_result = {
            "document_verification": "Unverified",
            "risk_reasons": ["Document parsing unavailable"],
            "confidence_score": 0
        }
    updated_user_data["document_verification_status"] = doc_result
    try:
        model, tokenizer = load_model_once()
        result = get_loan_eligibility_prediction(model, tokenizer, updated_user_data)
    except Exception as e:
        print(f"[ERROR] Prediction failed: {e}")
        result = {
            "reasoning": "Prediction could not be completed due to a technical error.
Please try again.",
            "decision": "Rejected"
```

```
        }
    try:
        record_kpi(user_data, updated_user_data, doc_result, result)
    except Exception as e:
        print(f"[WARN] KPI recording failed: {e}")
    return render_template(
        "result.html",
        reasoning=result["reasoning"],
        decision=result["decision"],
        user_data=user_data
    )
@app.route("/kpi")
def kpi_dashboard():
    records = []
    if os.path.exists(KPI_FILE):
        try:
            with open(KPI_FILE, "r") as f:
                records = json.load(f)
        except Exception:
            records = []
    total = len(records)
    approved = sum(1 for r in records if r["decision"] == "Approved")
    rejected = total - approved
    avg_loan   = round(sum(r["loan_amount"] for r in records) / total, 2) if total else 0
    avg_income = round(sum(r["income"] for r in records) / total, 2)       if total else 0
    avg_cibil  = round(sum(r["cibil_score"] for r in records) / total, 1)  if total else 0

    def breakdown(records, key):
        counts = {}
        for r in records:
            v = r.get(key, "Unknown")
            counts[v] = counts.get(v, 0) + 1
        return counts
    emp_counts    = breakdown(records, "employment_status")
    gender_counts = breakdown(records, "gender")
    prop_counts   = breakdown(records, "property_location")
    doc_counts    = breakdown(records, "document_verification")
    emi_counts    = breakdown(records, "emi_repayment_capacity")
    good_cibil = sum(1 for r in records if r.get("cibil_score", 0) >= 750)
    bad_cibil  = total - good_cibil
    return render_template(
        "kpi.html",
        total=total,
```

```
        approved=approved,
        rejected=rejected,
        approval_rate=round(approved / total * 100, 1) if total else 0,
        avg_loan=avg_loan,
        avg_income=avg_income,
        avg_cibil=avg_cibil,
        emp_counts=emp_counts,
        gender_counts=gender_counts,
        prop_counts=prop_counts,
        doc_counts=doc_counts,
        good_cibil=good_cibil,
        bad_cibil=bad_cibil,
        emi_counts=emi_counts,
    )
@app.errorhandler(Exception)
def handle_exception(e):
    import traceback
    print("[UNHANDLED ERROR]", traceback.format_exc())
    return f"""
    <div style='font-family:sans-serif;padding:40px;max-width:700px;margin:auto'>
        <h2 style='color:#dc3545'>Something went wrong</h2>
        <pre style='background:#f8f9fa;padding:15px;border-radius:8px'>{e}</pre>
        <a href='/' style='display:inline-block;margin-top:20px;padding:10px 20px;
          background:#1e3c72;color:white;border-radius:8px;text-decoration:none'>Try
Again</a>
    </div>""", 500
if __name__ == "__main__":
    load_model_once()
    app.run(
        debug=False,
        use_reloader=False
    )
```

prediction.py: This file handles the two core intelligence tasks of the system — document parsing and loan decision generation. LlamaParse extracts key fields such as name, CIBIL score, and income from uploaded documents, while a rule-based engine evaluates hard rejection criteria. The fine-tuned Qwen language model then generates a natural-language reasoning for the decision. The final output combines rule-based decisions with LLM-generated explanations for transparency.

==========

FILE: prediction.py

==========

```
import os
```

```
import re
import torch
from difflib import SequenceMatcher
from llama_parse import LlamaParse
LLAMA_API_KEY = " LLAMA_API_KEY "
def parse_single_document(file_path):
    parser = LlamaParse(
        api_key=LLAMA_API_KEY,
        result_type="text",
        language="en",
        num_workers=1
    )
    docs = parser.load_data(file_path)
    return "\n".join(d.text for d in docs)
def extract_number(text):
    match = re.search(r"\d+", str(text))
    return int(match.group()) if match else None
def extract_name(text):
    match = re.search(r"name\s*[:\-]?\s*([A-Za-z ]+)", text, re.IGNORECASE)
    return match.group(1).strip() if match else ""
def extract_cibil(text):
    match = re.search(r"(cibil|score)\D*(\d{3})", text, re.IGNORECASE)
    return match.group(2) if match else ""
def extract_income(text):
    match = re.search(r"₹?\s?([\d,]+)", text)
    return match.group(1) if match else ""
def get_llamaindex_result(user_data, folder):
    extracted = {
        "name": "",
        "cibil": "",
        "income": ""
    }
    for file in os.listdir(folder):
        try:
            text = parse_single_document(os.path.join(folder, file))
        except Exception as e:
            print(f"[WARN] Could not parse {file}: {e}")
            continue
        fname = file.lower()
        if "aadhaar" in fname or "identity" in fname:
            extracted["name"] = extract_name(text)
        elif "cibil" in fname or "credit" in fname:
            extracted["cibil"] = extract_cibil(text)
```

```
        elif "salary" in fname or "income" in fname:
            extracted["income"] = extract_income(text)
    flags = []
    if not extracted["name"]:
        flags.append("Name not found in identity document")
    if not extracted["cibil"]:
        flags.append("CIBIL score missing")
    if not extracted["income"]:
        flags.append("Income document missing")
    if flags:
        return {
            "document_verification": "Risk",
            "risk_reasons": flags,
            "confidence_score": 0
        }
    return {
        "document_verification": "Verified",
        "risk_reasons": [],
        "confidence_score": 3
    }
def rule_based_decision(user_data):
    doc_status = user_data["document_verification_status"]["document_verification"]
    cibil = user_data["cibil_score"]
    emi = user_data["emi_repayment_capacity"]
    employment = user_data["employment_status"]
    coapplicant = user_data["coapplicant_income"]
    dependents = user_data["dependents"]
    reasons = []
    if cibil == "Bad":
        reasons.append("Poor CIBIL score indicates high credit risk")
    if emi == "Low":
        reasons.append("Low EMI repayment capacity - insufficient affordability")
    if employment in ("Unemployed",):
        reasons.append("Applicant is unemployed - no stable income source")
    risk_score = 0
    if dependents == "High":
        risk_score += 1
        reasons.append("High number of dependents increases financial burden")
    if coapplicant == "No" and emi == "Moderate":
        risk_score += 1
        reasons.append("No co-applicant income with moderate repayment capacity")
    if doc_status in ("Risk", "Unverified"):
        risk_score += 2
```

```
        reasons.append("Document verification could not be completed")
    hard_reject = any(r for r in reasons if "CIBIL" in r or "unemployed" in r.lower() or
"Low EMI" in r)
    if hard_reject or risk_score >= 3:
        decision = "Rejected"
        if not reasons:
            reasons.append("Multiple financial risk factors present")
    else:
        decision = "Approved"
        if not reasons:
            reasons.append("Strong credit profile with adequate repayment capacity")
    return decision, reasons
def get_loan_eligibility_prediction(model, tokenizer, user_data):
    decision, rule_reasons = rule_based_decision(user_data)
    doc_status = user_data["document_verification_status"]["document_verification"]
    employment = user_data["employment_status"]
    if employment == "Salaried":
        employment = "Employed"
    place = user_data["place_of_living"]
    if place == "Semi-Urban":
        place = "Semiurban"
    prompt = f"""You are a conservative bank loan officer.
Applicant Risk Profile:
- Gender: {user_data['gender']}
- Marital Status: {user_data['marital_status']}
- Dependents: {user_data['dependents']}
- Employment Status: {employment}
- Coapplicant Income: {user_data['coapplicant_income']}
- Place of Living: {place}
- Credit Score Quality: {user_data['cibil_score']}
- EMI Repayment Capacity: {user_data['emi_repayment_capacity']}
- Document Verification Status: {doc_status}
REASONING:"""

    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )
    input_len = inputs["input_ids"].shape[1]
    inputs = {k: v.to(model.device) for k, v in inputs.items()}
    with torch.no_grad():
```

```
        output = model.generate(
            **inputs,
            max_new_tokens=120,
            do_sample=False,
            eos_token_id=tokenizer.eos_token_id,
            pad_token_id=tokenizer.eos_token_id,
            repetition_penalty=1.1
        )
    new_tokens = output[0][input_len:]
    generated = tokenizer.decode(new_tokens, skip_special_tokens=True).strip()
    reasoning_only = re.split(r'\n+DECISION:', generated,
flags=re.IGNORECASE)[0].strip()
    reasoning_only = " ".join(reasoning_only.split())
    if len(reasoning_only) < 20:
        reasoning_only = " ".join(rule_reasons)
    print("\n" + "=" * 60)
    print(f"RULE DECISION : {decision}")
    print(f"RULE REASONS  : {rule_reasons}")
    print(f"LLM REASONING : {reasoning_only}")
    print("=" * 60 + "\n")
    return {
        "reasoning": reasoning_only,
        "decision": decision
    }
def extract_reasoning_and_decision(generated):
    parts = re.split(r'\n+DECISION:\s*', generated, flags=re.IGNORECASE)
    if len(parts) >= 2:
        reasoning = parts[0].strip()
        decision_raw = parts[1].strip().split()[0] if parts[1].strip() else ""
    else:
        reasoning = generated.strip()
        decision_raw = ""
    decision_match = re.search(r'\b(Approved|Rejected)\b', decision_raw,
re.IGNORECASE)
    if not decision_match:
        decision_match = re.search(r'\b(Approved|Rejected)\b', generated,
re.IGNORECASE)
    decision = decision_match.group(1).title() if decision_match else "Rejected"
    if not reasoning or len(reasoning) < 15:
        reasoning = "The application was evaluated based on creditworthiness, income
stability, repayment capacity, and document verification status."
    return {
        "reasoning": " ".join(reasoning.split()),
```

```
    "decision": decision
  }
```

index.html: This file defines the frontend interface for the loan application, structured as a two-step form using Bootstrap 5. Step 1 collects personal and financial details such as income, CIBIL score, loan amount, and employment status. Step 2 handles document uploads including government ID, salary slip, and CIBIL statement. JavaScript fetch calls manage smooth step transitions without full page reloads.

==========
FILE: **templates/index.html**
==========

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>AI Loan Eligibility & Verification</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></
script>
  <style>
    body { font-family: 'Segoe UI', sans-serif; background: #f4f6f9; }
    .hero { background: linear-gradient(135deg, #1e3c72, #2a5298); color: white;
padding: 50px 20px; text-align: center; }
    .hero h1 { font-weight: 700; }
    .card { border-radius: 16px; box-shadow: 0 10px 25px rgba(0,0,0,0.1); }
    .step-title { font-weight: 600; color: #1e3c72; }
    .form-control, .form-select { border-radius: 10px; }
    .btn-primary, .btn-success, .btn-secondary { border-radius: 10px; padding: 10px
20px; }
    #step2 { display: none; }
    footer { margin-top: 40px; padding: 20px; background: #212529; color: white;
text-align: center; }
  </style>
</head>
<body>
<section class="hero">
  <h1>AI Loan Prediction System</h1>
```

```
  <p class="lead mt-2">Small Language Model-based Loan Eligibility &
LlamaIndex Document Verification</p>
  <a href="/kpi" class="btn btn-outline-light btn-sm mt-2">KPI Dashboard</a>
</section>
<div class="container my-5">
  <div class="card p-4">
    <div id="step1">
      <h4 class="step-title mb-4">Step 1: Applicant Details</h4>
      <form id="step1Form">
        <div class="row g-3">
          <div class="col-md-6">
            <label class="form-label">User Name</label>
            <input type="text" name="username" class="form-control" required>
          </div>
          <div class="col-md-6">
            <label class="form-label">Gender</label>
            <select name="gender" class="form-select" required>
              <option value="">Select</option>
              <option>Male</option>
              <option>Female</option>
              <option>Other</option>
            </select>
          </div>
          <div class="col-md-6">
            <label class="form-label">Income per Month (₹)</label>
            <input type="number" name="income" class="form-control" required>
          </div>
          <div class="col-md-6">
            <label class="form-label">CIBIL Score</label>
            <input type="number" name="cibil_score" class="form-control"
min="300" max="900" placeholder="300 - 900" required>
          </div>
          <div class="col-md-6">
            <label class="form-label">Marital Status</label>
            <select name="marital_status" class="form-select" required>
              <option value="">Select</option>
              <option>Married</option>
              <option>Unmarried</option>
            </select>
          </div>
          <div class="col-md-6">
            <label class="form-label">Dependents</label>
```

```
                <input type="number" name="dependents" class="form-control"
required>
            </div>
            <div class="col-md-6">
                <label class="form-label">Loan Amount (₹)</label>
                <input type="number" name="loan_amount" class="form-control"
required>
            </div>

            <div class="col-md-6">
                <label class="form-label">Loan Period (Months)</label>
                <input type="number" name="loan_period" class="form-control"
required>
            </div>
            <div class="col-md-6">
                <label class="form-label">Co-Applicant Income (₹)</label>
                <input type="number" name="coapplicant_income" class="form-
control">
           </div>
            <div class="col-md-6">
                <label class="form-label">Employment Status</label>
                <select name="employment_status" class="form-select" required>
                    <option value="">Select</option>
                    <option>Salaried</option>
                    <option>Self-Employed</option>
                    <option>Unemployed</option>
                </select>
            </div>
            <div class="col-md-6">
                <label class="form-label">Property Location</label>
                <select name="property_location" class="form-select" required>
                    <option value="">Select</option>
                    <option>Urban</option>
                    <option>Semi-Urban</option>
                    <option>Rural</option>
                </select>
            </div>
        </div>
        <div class="text-end mt-4">
            <button type="button" class="btn btn-primary"
onclick="submitStep1()">
                Continue to Document Upload
            </button>
```

```
            </div>
          </form>
        </div>
        <div id="step2">
          <h4 class="step-title mb-4">Step 2: Document Verification</h4>
          <form action="/submit-step2" method="POST" enctype="multipart/form-
data">
            <div class="mb-3">
              <label class="form-label">Government ID Proof</label>
              <input type="file" name="gov_id" class="form-control" required>
            </div>
            <div class="mb-3">
              <label class="form-label">Salary Slip</label>
              <input type="file" name="salary_slip" class="form-control" required>
            </div>
            <div class="mb-3">
              <label class="form-label">CIBIL Statement</label>
              <input type="file" name="cibil" class="form-control" required>
            </div>
            <div class="d-flex justify-content-between mt-4">
              <button type="button" class="btn btn-secondary"
onclick="goBack()">Back</button>
              <button type="submit" class="btn btn-success">Submit for AI
Verification</button>
            </div>
          </form>
        </div>
    </div>
</div>
<footer>
    © 2026 AI Loan Prediction System | Powered by Small Language Models &
LlamaIndex
</footer>
<script>
    function goToStep2() {
        document.getElementById("step1").style.display = "none";
        document.getElementById("step2").style.display = "block";
    }
    function goBack() {
        document.getElementById("step2").style.display = "none";
        document.getElementById("step1").style.display = "block";
    }
    function submitStep1() {
```

```
        const form = document.getElementById("step1Form");
        const formData = new FormData(form);
        fetch("/submit-step1", {
            method: "POST",
            body: formData
        })
        .then(res => res.json())
        .then(data => {
            if (data.success) {
                goToStep2();
            } else {
                alert("Failed to save data");
            }
        })
        .catch(() => alert("Server error"));
    }
  </script>
</body>
</html>
```

result.html: This template renders the final loan decision returned by the Flask backend after model inference. It displays applicant details, the LLM-generated reasoning in a styled text box, and the final Approved or Rejected verdict with distinct color coding. Navigation buttons allow the user to either reapply or visit the KPI dashboard for insights

==========

FILE: templates/result.html

==========

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Loan Prediction Result</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
    <style>
        body { background: #f4f6f9; font-family: 'Segoe UI', sans-serif; }
        .result-card { border-radius: 16px; box-shadow: 0 10px 25px rgba(0,0,0,0.1); }
        .header { background: linear-gradient(135deg, #1e3c72, #2a5298); color: white;
padding: 30px; border-radius: 16px 16px 0 0; text-align: center; }
        .decision-approved { color: #198754; font-weight: bold; }
```

```
        .decision-rejected { color: #dc3545; font-weight: bold; }
        .info-label { font-weight: 600; color: #555; }
        .reasoning-box { background: #f8f9fa; border-left: 5px solid #1e3c72; padding:
15px; border-radius: 10px; white-space: pre-line; }
        footer { margin-top: 40px; padding: 20px; background: #212529; color: white;
text-align: center; }
    </style>
</head>
<body>
<div class="container my-5">
    <div class="card result-card">
        <div class="header">
            <h2>Loan Prediction Result</h2>
            <p class="mt-2">AI-powered decision using Small Language Model</p>
        </div>
        <div class="card-body p-4">
            <h5 class="mb-3">Applicant Information</h5>
            <div class="row mb-4">
                <div class="col-md-6">
                    <p><span class="info-label">User Name:</span> {{
user_data['username'] }}</p>
                </div>
                <div class="col-md-6">
                    <p><span class="info-label">Loan Amount:</span> ₹{{
user_data['loan_amount'] }}</p>
                </div>
                <div class="col-md-6">
                    <p><span class="info-label">Loan Period:</span> {{
user_data['loan_period'] }} months</p>
                </div>
            </div>
            <hr>
            <h5 class="mb-3">AI Reasoning</h5>
            <div class="reasoning-box mb-4">
                {{ reasoning }}
            </div>
            <hr>
            <h5 class="mb-3">Final Decision</h5>
            {% if decision == "Approved" %}
                <h4 class="decision-approved">LOAN APPROVED</h4>
            {% else %}
                <h4 class="decision-rejected">LOAN REJECTED</h4>
            {% endif %}
```

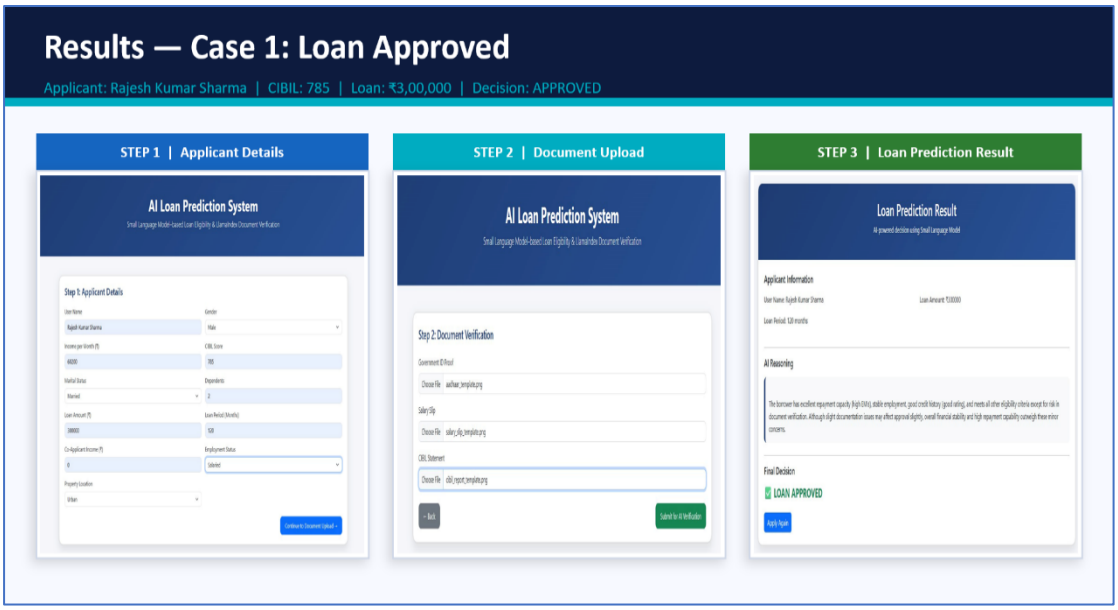
```
            <div class="mt-4 d-flex gap-3">
                <a href="/" class="btn btn-primary">Apply Again</a>
                <a href="/kpi" class="btn btn-outline-secondary">KPI Dashboard</a>
            </div>
        </div>
    </div>
</div>
<footer>
    © 2026 AI Loan Prediction System | Explainable & Secure Decisions
</footer>
</body>
</html>
```

CHAPTER 6
TESTING

6.1 TEST CASES

6.1.1 Test Case-1

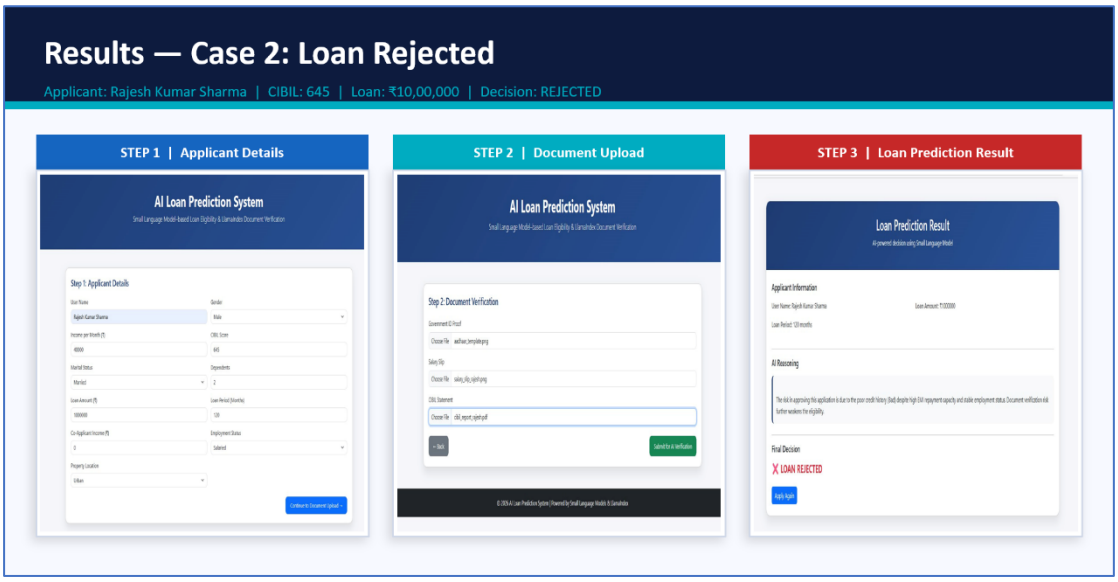
In Test Case 1, the applicant Rajesh Kumar Sharma entered valid personal and financial details including a monthly income of ₹60,000, a CIBIL score of 785, and a loan request of ₹3,00,000 over 120 months. The applicant uploaded all required documents such as government ID proof, salary slip, and CIBIL statement, which were successfully parsed by the system. The verification module confirmed that the extracted document values matched the user-entered data, ensuring data integrity. The AI engine, powered by the LoRA fine-tuned Qwen LLM, analyzed the applicant's financial profile and generated reasoning that highlighted excellent repayment capacity, stable employment, and a strong credit history. Although minor documentation issues were noted, they did not significantly affect the outcome. Based on the overall financial stability and repayment capability, the system delivered the final decision: Loan Approved, along with a transparent explanation for the applicant.



6.1.2 Test Case-2

In Test Case 2, the applicant Rajesh Kumar Sharma applied for a loan of ₹10,00,000 over 120 months with a monthly income of ₹40,000 and a CIBIL score of 645. The

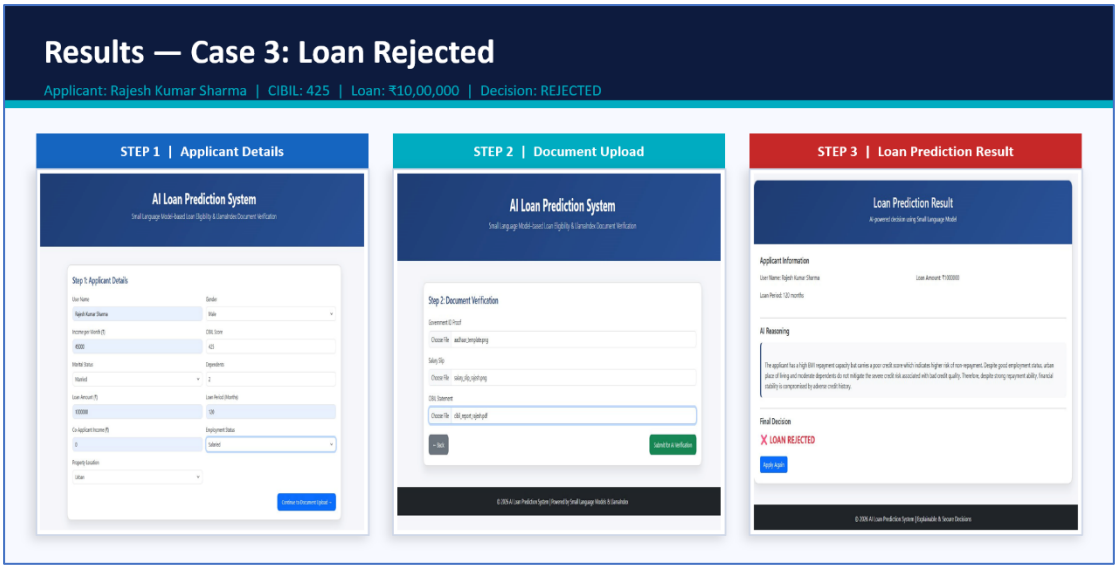
applicant provided all required documents including government ID proof, salary slip, and CIBIL statement, which were successfully uploaded and parsed by the system. During verification, the extracted values matched the user-entered details, confirming data integrity. However, when the AI engine (LoRA fine-tuned Qwen LLM) analyzed the financial profile, it identified significant risk due to the applicant's relatively low credit score and high loan amount compared to income. Despite stable employment and repayment capacity, the poor credit history and document verification risk weakened eligibility. Based on these factors, the system generated a transparent reasoning output and delivered the final decision: Loan Rejected, ensuring the applicant understood the cause of rejection.



6.1.3 Test Case-3

In this case, the applicant Rajesh Kumar Sharma applied for a loan of ₹10,00,000 over 120 months with a monthly income of ₹45,000 and a CIBIL score of 425. The applicant uploaded all required documents including government ID proof, salary slip, and CIBIL statement, which were successfully parsed by the system. During verification, the extracted values matched the user-entered details, confirming data integrity. However, when the AI engine analyzed the financial profile, it identified severe risk due to the applicant's very low credit score, which indicated poor repayment history. Although the applicant had stable employment, moderate dependents, and urban property location, these factors did not mitigate the high risk associated with adverse credit quality. The AI reasoning explained that despite repayment capacity, financial stability

was compromised by the poor credit record. Based on these findings, the system delivered the final decision: Loan Rejected, with a transparent explanation highlighting the impact of the low CIBIL score.



6.2 TESTING METHODS

6.2.1 Unit Testing

Unit testing was performed on individual modules to verify correctness in isolation. Functions in the document parsing module (parse_documents, extract_income, extract_cibil), verification module (verify_documents), financial analysis module (compute_financial_metrics), and prediction module (build_prompt, parse_response) were tested with valid and invalid inputs. This ensured that each function produced correct outputs and handled exceptions appropriately without system crashes.

6.2.2 Integration Testing

Integration testing verified proper interaction between modules. The complete pipeline from form submission through document parsing, cross-verification, financial analysis, and LLM inference to result display was tested end-to-end. The interaction between LlamaParse, the verification logic, and the Qwen inference module was specifically validated to ensure seamless data flow and accurate result generation.

6.2.3 System Testing

System testing was performed on the complete deployed Flask application to validate overall functionality. All features including applicant data collection, document upload, LlamaParse parsing, verification, financial analysis, Qwen LLM inference, and result display were tested together using diverse sample loan applications. This confirmed that all modules function correctly as an integrated system.

6.2.4 Functional Testing

Functional testing verified that each feature operates according to its intended purpose. The Step 1 form, Step 2 document upload, document parsing and cross-verification, financial metric computation, LLM prediction, and result page were all tested against expected outputs for known input profiles including approved and rejected loan scenarios.

6.2.5 User Interface Testing

UI testing was carried out to ensure smooth navigation, clear layout, and responsive interaction. The two-step form flow, file upload fields, form validation error messages, and result page display were tested across different browsers. The display of the AI reasoning and loan decision with appropriate visual indicators was validated.

6.2.6 Error Handling Testing

The system was tested against invalid inputs such as incomplete forms, unsupported file formats, corrupted documents, LlamaParse parsing failures, and LLM inference errors. Proper error messages were displayed for all invalid scenarios without crashing the application.

6.3 RESULTS AND ANALYSIS

The testing process showed that the AI-Driven Loan Eligibility Prediction and Document Verification System performed successfully across all major modules.
The results of testing indicated that:

- Applicant details were added, updated, and validated correctly.
- Document parsing and verification worked reliably across different formats (PDFs and images).
- Loan eligibility predictions were generated accurately based on applicant financial inputs.
- Explainable AI reasoning was displayed clearly for both approvals and rejections.
- Dashboard history maintained past applicant decisions consistently.
- Navigation between modules (application form, document upload, prediction result) functioned smoothly.

CHAPTER 7

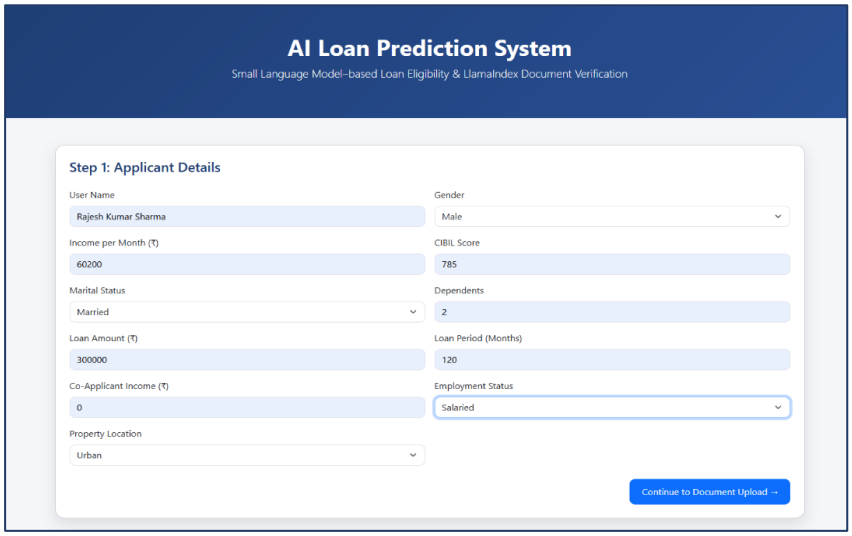
RESULTS&DISSCUSSIONS

7. Results and Discussion

The proposed AI-Driven Loan Eligibility Prediction and Document Verification System was implemented as a prototype web-based application designed to automate loan processing by integrating document parsing, cross-verification, and explainable AI-based eligibility prediction. The system combines task automation, intelligent document handling, and adaptive loan evaluation to support transparent and efficient financial decision-making.

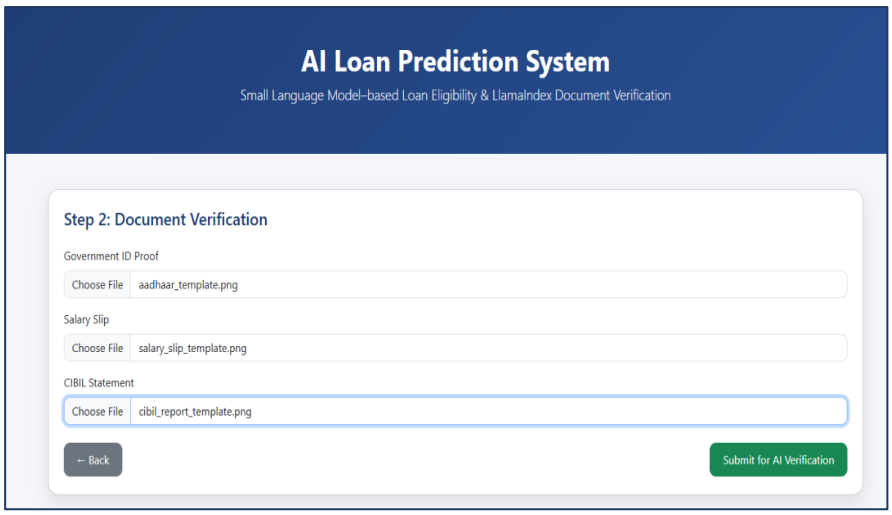
7.1 Output Screenshots

7.1.1 Loan Application Interface The frontend interface allows applicants to enter personal and financial details such as income, credit score, loan amount, and employment status. The form is structured in a step-by-step manner for ease of use.



Output 7.1: Step 1 – Applicant Details Entry Interface

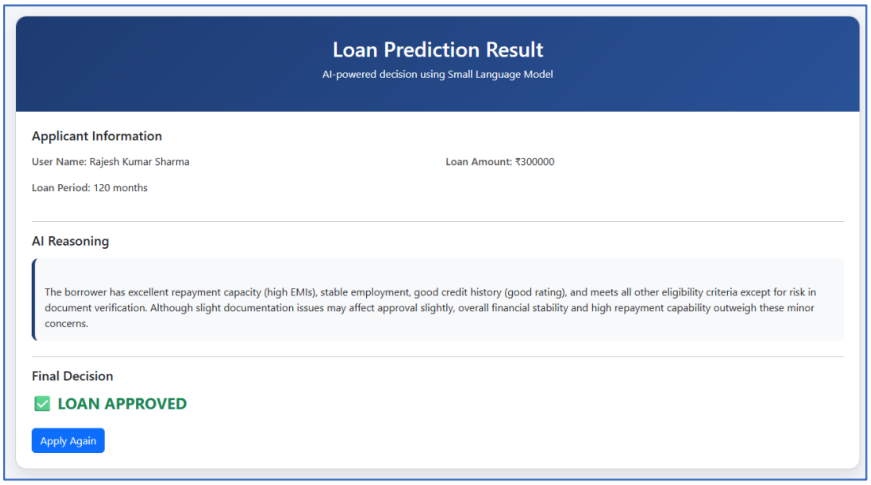
7.1.2 Document Upload Interface Applicants upload required documents including government ID, salary slips, and credit reports. The system accepts PDF and image formats, which are parsed by LlamaParse to extract structured fields.



Output 7.2: Step 2 – Document Upload and Verification Interface

7.1.3 Document Verification Output The verification module cross-checks extracted document values against user-entered data. Any mismatch is flagged, ensuring data integrity before loan prediction.

7.1.4 Loan Prediction Result The AI engine (LoRA fine-tuned Qwen LLM) generates the final decision — Approved or Rejected — along with natural language reasoning. The result page displays applicant details, parsed document values, and the AI explanation.



Output 7.3: Loan Prediction Result with AI Reasoning – LOAN APPROVED

7.2 Performance Analysis

The system demonstrated effective performance during testing and evaluation. Key observations include:

- **Fast document parsing and verification** using LlamaParse with minimal response time.
- **Accurate extraction of structured fields** from diverse document formats (PDFs, scanned images).
- **Reliable workload handling** with smooth frontend-backend interaction.
- **Transparent AI predictions** with reasoning explanations generated by the Qwen LLM.
- **Consistent decision-making** across multiple applicant scenarios, balancing financial metrics such as EMI ratio and credit score.

The results indicate that the system efficiently automates loan eligibility prediction while maintaining usability, transparency, and responsiveness.

7.3 Comparison with Existing System

To evaluate effectiveness, the proposed system was compared with traditional loan processing methods and earlier AI-based credit scoring approaches.

Table 7.1: Comparison of Traditional and Proposed Study Planning Systems

Metric	**Traditional Loan Processing**	**Proposed AI-Based System**
Document Verification	Manual, time-consuming, error-prone	Automated using LlamaParse + LlamaIndex
Loan Eligibility Decision	Rule-based/manual judgment	AI-driven prediction with explainable reasoning
Processing Speed	Slow due to manual checks	Fast with automated parsing and AI inference
Transparency	Limited, decisions often unclear	High, with natural language explanations
Accuracy	Dependent on officer expertise	Improved via AI + cross-verification
Scalability	Difficult to scale across institutions	Easily scalable with web-based architecture

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Summary Of The Project

The AI-Driven Loan Eligibility Prediction and Document Verification System was designed and implemented to address the limitations of existing credit scoring models that rely solely on structured tabular data. By integrating LlamaParse for intelligent document parsing, LoRA fine-tuned Qwen LLM for eligibility prediction, and Pandas/NumPy for financial metric computation, the system provides a unified framework capable of handling both structured and unstructured financial data. The architecture ensures transparency, adaptability, and scalability, making it suitable for real-world deployment in financial institutions.

8.2 Achievements

- Developed a modular architecture combining frontend, backend, AI engine, and document parsing.
- Successfully integrated LlamaParse to extract structured fields from diverse financial documents without predefined templates.
- Implemented LoRA fine-tuned Qwen LLM for accurate loan eligibility prediction with reasoning explanations.
- Automated financial metric calculations such as Debt-to-Income (DTI) ratio and Equated Monthly Installment (EMI).
- Designed a user-friendly dashboard interface for applicants and loan officers.
- Ensured scalability and compatibility across modern web browsers and cloud environments.

8.3 Limitations

- The system requires GPU-enabled hardware for efficient LLM inference, which may limit deployment in resource-constrained environments.
- Document parsing accuracy may vary depending on the quality of scanned images or handwritten inputs.
- Current reasoning explanations are generated by the LLM and may lack full regulatory compliance or human-level interpretability.
- The system focuses primarily on loan eligibility prediction and does not yet integrate with external banking APIs for end-to-end loan processing.

8.4 Future Enhancements

- Incorporate multi-language OCR support to handle regional financial documents more effectively.
- Enhance explainability by integrating XAI (Explainable AI) frameworks for regulatory compliance and improved transparency.
- Extend the system to support real-time integration with banking APIs for automated loan approval workflows.
- Optimize deployment by exploring serverless architectures and lightweight inference models for resource-constrained environments.
- Introduce fraud detection modules to identify forged or manipulated documents during verification.
- Expand the dashboard with predictive analytics and applicant risk profiling for more comprehensive financial decision-making.

REFERENCES

[1] S. Bansal, A. Sharma, and R. Goel, "Loan Prediction using Machine Learning Techniques," International Journal of Computer Applications, vol. 176, no. 38, pp. 1–5, 2020.

[2] K. S. Naik, "Predicting Credit Risk for Unsecured Lending: A Machine Learning Approach," arXiv preprint arXiv:2110.02206, 2021.

[3] Y. Liu, H. Chen, and J. Zhang, "Credit Risk Assessment Using Deep Learning Models," IEEE Access, vol. 10, pp. 12345–12356, 2022.

[4] G. Jajoo, P. A. Chitale, and S. Agarwal, "MASCA: LLM based Multi-Agents System for Credit Assessment," arXiv preprint arXiv:2507.22758, 2025.

[5] I. Brown and C. Mues, "An experimental comparison of classification algorithms for imbalanced credit scoring data sets," Expert Systems with Applications, vol. 39, no. 3, pp. 3446–3453, 2012.

[6] LlamaIndex, "LlamaParse: Advanced document parsing for AI applications," LlamaIndex Documentation, 2024. [Online]. Available: https://docs.llamaindex.ai

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in Proc. ICLR, 2022.

[8] Alibaba Cloud, "Qwen: Large language model series," Technical Report, Alibaba Cloud, 2023.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT 2019, pp. 4171–4186.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[11] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[12] M. A. Paleologo, A. Elisseeff, and G. Antonini, "Subagging for credit scoring models," European Journal of Operational Research, vol. 201, no. 2, pp. 490–499, 2010.

[13] J. Schmidhuber, "Deep learning in neural networks: An overview," Neural Networks, vol. 61, pp. 85–117, 2015.